



Université Mohammed Premier
Faculté des Sciences d'Oujda
Département d'Informatique



Université Mohammed Premier — Oujda
Faculté des Sciences / Département d'Informatique

RAPPORT DE PROJET : DATA MINING

Master spécialisé en Ingénierie Informatique

**Système de Prédiction de Mots Basé sur
les N-grammes et l'Algorithme Apriori**

Encadrant : Prof. AZZEDDINE MAZRAOUI

Présenté par : TAOUFIK MANSOURI

ES-SADKIA SAMI

Année universitaire : 2025–2026

Remerciements

Nous tenons à exprimer, en premier lieu, notre profonde gratitude à notre encadrant au sein du Département d'Informatique de la Faculté des Sciences de l'Université Mohammed Premier. Ses conseils avisés, sa disponibilité constante et la qualité de son suivi pédagogique ont été déterminants pour orienter nos choix techniques et théoriques tout au long de ce projet.

Nous remercions chaleureusement l'ensemble du corps enseignant du Master 2 Informatique pour la qualité de l'enseignement reçu, en particulier les cours de Data Mining, d'algorithmique avancée et de traitement du langage naturel qui ont constitué le socle théorique de ce travail.

Nos remerciements s'adressent également aux responsables des infrastructures informatiques de la faculté pour l'accès aux serveurs et aux ressources de calcul nécessaires à l'exécution des expériences décrites dans ce rapport.

Enfin, nous remercions nos camarades de promotion pour les discussions enrichissantes, les sessions de débogage collaboratives et l'émulation intellectuelle mutuelle qui ont rendu ce projet particulièrement formateur.

Sommaire

Introduction	1
Chapitre 1 : Contexte, problématique et état de l’art	3
1.1 La prédiction de mots — définition et enjeux	3
1.2 Approches existantes en traitement du langage naturel	4
1.3 L’apport du Data Mining au NLP	6
Chapitre 2 : Fondements théoriques des algorithmes utilisés	8
2.1 Les modèles de N-grammes	8
2.2 L’algorithme Apriori et les règles d’association	11
2.3 Adaptation de l’Apriori à la prédiction séquentielle	14
Chapitre 3 : Conception, implémentation et résultats	17
3.1 Architecture générale du système	17
3.2 Implémentation détaillée en Java / Spring Boot	19
3.3 Interface utilisateur et intégration REST	24
3.4 Résultats, évaluation et discussion	25
Conclusion et perspectives	28
Bibliographie	30
Annexes	32
Table des matières	34

Introduction

Taper un message sur son smartphone, lancer une recherche sur internet ou rédiger un document sur ordinateur : ces gestes sont devenus si fluides qu'on en oublie presque la technologie qui les accompagne. Pourtant, derrière cette apparente simplicité, des systèmes complexes s'efforcent constamment de deviner nos intentions pour nous faire gagner du temps. C'est le rôle de l'autocomplétion et de la prédiction textuelle. Aujourd'hui, ce domaine est largement dominé par des modèles d'intelligence artificielle gigantesques. Si ces réseaux de neurones sont impressionnants, ils s'avèrent aussi extrêmement gourmands en énergie et en ressources. C'est ce constat qui nous a poussés à explorer une autre voie : celle d'une solution plus légère, rapide et accessible, s'appuyant sur des approches statistiques et logiques éprouvées. C'est dans cette perspective que s'inscrit notre travail. Pour relever ce défi, nous avons choisi de concevoir et de développer notre propre moteur de prédiction. Développée en Java avec le framework Spring Boot, notre solution fait collaborer deux approches complémentaires : d'un côté, l'analyse de séquences courtes (les bi-grammes et trigrammes) pour capter le contexte immédiat de la phrase ; de l'autre, l'algorithme Apriori, que nous avons détourné de son usage classique pour extraire de véritables règles d'association logique entre les mots. Tout au long de ce projet, nous tenterons ainsi de répondre à une problématique centrale : Comment exploiter l'algorithme Apriori sur des suites de mots afin de mettre en place un système de prédiction textuelle qui soit à la fois efficace, pertinent et facile à déployer sur le web ? Pour guider le lecteur dans notre démarche, ce document s'articule autour de trois grands chapitres. Le premier posera les bases théoriques en dressant un état de l'art de la prédiction textuelle et du Traitement du Langage Naturel (NLP). Le deuxième chapitre détaillera les fondements mathématiques et logiques qui structurent les N-grammes et l'algorithme Apriori. Enfin, le dernier chapitre sera consacré à notre réalisation concrète : nous y présenterons l'architecture de notre outil, nos choix de programmation et une analyse critique des résultats obtenus.

Chapitre 1

Contexte, problématique et état de l’art

1.1 Mieux comprendre la prédiction de mots

En termes très simples, la prédiction de mots consiste à faire deviner à une machine quel mot l'utilisateur est sur le point d'écrire, en s'appuyant sur ce qu'il a déjà tapé auparavant. Dans la littérature anglophone, c'est ce qu'on appelle la *next-word prediction*. Le principe fondamental s'appuie sur des probabilités pour estimer les chances qu'un mot succède à un autre dans une langue bien précise¹.

1.1.1 Où retrouve-t-on cette technologie ?

Les exemples d'utilisation de cette technologie foisonnent dans nos outils numériques quotidiens :

- **Sur nos smartphones** : les claviers intelligents comme SwiftKey ou Gboard nous proposent sans arrêt des mots pour qu'on puisse envoyer nos messages plus vite.
- **Dans nos recherches web** : dès que l'on tape un mot sur Google, une liste déroulante tente de deviner notre question complète.
- **Lors de la rédaction de documents** : des éditeurs comme Microsoft Word essaient désormais de compléter nos phrases pour nous faire gagner du temps.
- **Dans le milieu du handicap** : les outils d'autocomplétion sont d'une aide précieuse pour les personnes qui ont des difficultés à s'exprimer ou à manipuler un clavier classique.
- **Pour les développeurs** : même en programmant, des environnements comme VS Code complètent nos lignes de code toutes seules.

1.1.2 Les vrais défis à relever

Malgré les apparences, ce n'est pas si simple de construire un tel système. D'abord, on se heurte à ce qu'on appelle la **dispersion des données** (*data sparsity*). Pour faire court :

1. JURAFSKY Dan, MARTIN James H., *Speech and Language Processing*, 3^e éd., Pearson, 2024, p. 33.

il existe tellement de combinaisons de mots possibles que, même en analysant des milliers de livres, la majorité des phrases n'apparaîtra jamais. Ensuite, il y a le souci du **contexte à distance**. Parfois, pour deviner le bon mot, la machine doit se souvenir d'un sujet qui a été évoqué au tout début de la phrase. Enfin, il faut que l'ensemble réagisse à la milliseconde : l'utilisateur ne patientera jamais devant son écran pour attendre qu'une suggestion se charge.

1.2 Ce qui se fait déjà dans le domaine

1.2.1 L'ancienne méthode : les règles strictes

Aux prémices de l'informatique, on a tenté d'inculquer aux ordinateurs les règles de grammaire exactes, de façon très formelle. Mais on s'est assez vite rendu compte que la langue humaine est trop vivante, trop nuancée et truffée d'exceptions pour que cette approche "symbolique" fonctionne sur de longs textes variés.

1.2.2 L'arrivée des probabilités

C'est en comptant simplement les mots et en calculant des probabilités que les choses ont réellement avancé. Comme le font très justement remarquer Manning et Schütze : « *l'approche statistique s'est imposée comme la méthode la plus robuste pour le traitement de grands volumes de textes* »². C'est exactement de là que viennent les modèles de N-grammes, qui sont le point de départ de notre projet.

1.2.3 L'ère des réseaux de neurones

Plus récemment, depuis l'arrivée d'outils comme *Word2Vec* (autour de 2013), l'apprentissage profond a littéralement pris le relais. Puis, en 2017, la révolution des *Transformers* a donné naissance à des architectures surpuissantes comme BERT ou GPT³. C'est bluffant, mais le revers de la médaille c'est que ça demande des ordinateurs colossaux et des temps de calcul faramineux. Ce n'est donc pas du tout adapté quand on cherche une petite solution légère et économe.

2. MANNING Christopher D., SCHÜTZE Hinrich, *Foundations of Statistical Natural Language Processing*, MIT Press, 1999, p. 192.

3. VASWANI Ashish *et al.*, « Attention Is All You Need », in : *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.

1.3 Pourquoi lier le Data Mining et le texte ?

Habituellement, le Data Mining sert à fouiller dans de gigantesques bases de données pour y repérer des informations cachées ou des tendances⁴. L'idée d'utiliser cette discipline pour traiter des textes nous a semblé être une excellente alternative aux lourds réseaux de neurones.

1.3.1 Des paniers d'achats aux phrases

À la base, les règles d'association ont été inventées pour analyser les achats dans les supermarchés. Le but était de se dire : "Tiens, les clients qui achètent du beurre prennent souvent de la confiture". Nous avons fait le pari que ce même principe fonctionnerait pour les mots : "Si quelqu'un tape 'il' et 'fait', il y a de fortes chances pour qu'il écrive 'beau' juste après".

1.3.2 La notion d'ordre change tout

Il y a cependant un détail de taille. Contrairement à un caddie de supermarché où l'ordre des articles n'a strictement aucune importance, dans une phrase, l'ordre des mots veut tout dire ! L'originalité de notre projet a donc consisté à modifier le comportement habituel de l'algorithme Apriori pour qu'il respecte l'ordre dans lequel les mots s'enchaînent. C'est ce côté "séquentiel" qui fait tout l'intérêt de notre démarche, comme nous allons le voir dans le chapitre suivant.

4. AGGARWAL Charu C., *Machine Learning for Text*, Springer, 2018, p. 9.

Chapitre 2

Fondements théoriques des algorithmes utilisés

2.1 Un regard plus théorique sur les N-grammes

2.1.1 C'est quoi exactement un N-gramme ?

Pour faire simple, un N-gramme n'est rien d'autre qu'un bloc de mots qui se suivent dans un texte. Si on a une phrase quelconque, on la découpe en petits morceaux de taille N .

Concrètement, dans le cadre de notre projet, nous nous sommes focalisés sur deux tailles de blocs bien spécifiques :

- **Les bigrammes** (où $N = 2$) : on prend les mots deux par deux. Si la phrase est "je ne sais pas", on aura les paires $[je, ne]$, $[ne, sais]$, etc.
- **Les trigrammes** (où $N = 3$) : on prend les mots par groupes de trois, ce qui donne $[je, ne, sais]$.

2.1.2 La logique mathématique derrière tout ça

Ce système repose sur une idée très connue en mathématiques : l'hypothèse de Markov. Cette hypothèse part du principe que pour deviner ce qui va se passer, on a juste besoin de regarder ce qui vient de se produire. Donc, si on utilise un trigramme, on estime que le prochain mot dépend purement et simplement des deux mots qui le précèdent.

En pratique, on calcule cette probabilité en comptant bêtement combien de fois on a vu une suite de mots dans nos fichiers d'entraînement :

$$P(w_n | w_{n-1}) = \frac{C(w_{n-1}, w_n)}{C(w_{n-1})} \quad (\text{pour les bigrammes}) \quad (2.1)$$

$$P(w_n | w_{n-2}, w_{n-1}) = \frac{C(w_{n-2}, w_{n-1}, w_n)}{C(w_{n-2}, w_{n-1})} \quad (\text{pour les trigrammes}) \quad (2.2)$$

Ici, la lettre C représente tout simplement le nombre de fois où on a repéré cette combinaison de mots.

2.1.3 Le mur de la dispersion des données

Mais il y a un problème de taille. Si on prend un dictionnaire complet, le nombre de combinaisons possibles est gigantesque. Beaucoup de ces suites de mots ne se rencontreront jamais dans la vraie vie, ce qui nous donne plein de probabilités égales à zéro et rend le modèle inefficace.

Pour contourner ce problème agaçant, nous avons emprunté une technique à l'algorithme Apriori : on fixe un seuil minimum de présence, qu'on appelle le support. Dans notre code, si une suite de mots n'apparaît pas au moins 3 fois, on s'en débarrasse. C'est radical, mais ça permet de nettoyer les données de façon très efficace.

2.2 Le fameux algorithme Apriori

2.2.1 D'où vient-il ?

L'algorithme Apriori n'est pas récent. Il a été créé en 1994 par Agrawal et Srikant, au départ pour analyser les tickets de caisse dans les magasins¹. Sa logique s'appuie sur une règle toute simple (la propriété d'anti-monotonie) : si un groupe d'articles se vend très bien, alors on est sûr que chaque article de ce groupe se vend bien séparément. À l'inverse, si un article est invendable, aucun groupe contenant cet article ne marchera.

2.2.2 Comment savoir si une règle est pertinente ?

Pour savoir si on tient une bonne association, on s'appuie sur trois critères de notation :

1. AGRAWAL Rakesh, SRIKANT Ramakrishnan, « Fast Algorithms for Mining Association Rules », in : *Proceedings of the 20th International Conference on Very Large Data Bases*, Morgan Kaufmann, 1994, pp. 487–499.

TABLE 2.1 – Les trois critères d'évaluation d'une règle

Le critère	Ce que ça veut dire concrètement	Le calcul
Le Support	C'est la popularité globale : à quelle fréquence voit-on l'ensemble A et B dans nos données.	$supp(A \Rightarrow B) = \frac{ A \cup B }{N}$
La Confiance	C'est la fiabilité : quand j'ai A , à quel point suis-je sûr de voir aussi B ?	$conf(A \Rightarrow B) = \frac{ A \cup B }{ A }$
Le Lift	C'est l'effet de surprise : est-ce que A et B sont liés par une vraie logique, ou est-ce juste le hasard ?	$lift(A \Rightarrow B) = \frac{conf(A \Rightarrow B)}{P(B)}$

Quand notre programme doit choisir un mot à suggérer, il calcule un **score global** en multipliant la confiance par le lift (qu'on plafonne à 5 pour ne pas que les résultats explosent bêtement) :

$$score = conf(A \Rightarrow B) \times \min(lift(A \Rightarrow B), 5) \quad (2.3)$$

De plus, on donne un coup de pouce (un bonus $\times 1,5$) aux prédictions qui découlent de trigrammes, car on s'est rendu compte qu'elles étaient souvent beaucoup plus justes.

2.2.3 Le fonctionnement par étapes de l'Apriori

Traditionnellement, l'algorithme mouline en plusieurs étapes :

1. Il commence par lister les articles (ou les mots) individuels fréquents.
2. Ensuite, il combine ces articles pour former des paires, et il élimine toutes celles qui sont trop rares.
3. Puis, il regroupe par trois, et ainsi de suite.
4. À la fin, il tire de ces associations des règles de type "Si A alors B ", en supprimant celles dont la note de confiance est trop faible.

2.3 Faire marcher l'Apriori avec du texte

2.3.1 L'importance de l'ordre

Comme je l'évoquais un peu plus haut, le vrai défi a été d'adapter ce concept mathématique au texte. Dans un supermarché, acheter des pommes puis du lait, ou du lait puis des pommes, ça revient au même. Dans une phrase, c'est complètement différent !

Nous avons donc forcé notre algorithme à respecter un ordre strict. Au lieu de chercher des mots "en vrac", il doit chercher des bigrammes et des trigrammes bien collés les uns aux autres. Ainsi, dans la phrase «*je ne sais pas*», le programme est obligé de lire de gauche à droite : [je,ne], puis [ne,sais].

2.3.2 Les règles qu'on obtient au final

Grâce à cette petite astuce, on arrive à extraire des règles d'association qui ont du sens pour la langue :

- **À partir des bigrammes** : on obtient des règles directes comme "Si on écrit w_1 , le prochain mot sera sûrement w_2 ".
- **À partir des trigrammes** : on a des règles un peu plus pointues du genre "Si on vient de taper w_1 et w_2 , prépare-toi à voir w_3 ".

Ce sont précisément ces règles qui remplacent les gros calculs probabilistes qu'on ferait dans un système classique, mais ici elles sont déjà filtrées et triées par l'algorithme Apriori.

2.3.3 Nos petits réglages maison

Voici les paramètres précis que nous avons choisi d'intégrer dans le code :

TABLE 2.2 – Les réglages finaux de notre programme

Le réglage	La valeur	Pourquoi avoir choisi ça ?
minSupportCount	3 fois	Pour ne pas être pollué par une faute de frappe isolée
minConfiance	10%	Un seuil assez indulgent pour proposer un peu de diversité
Bonus trigramme	$\times 1,5$	Pour récompenser les contextes de phrase plus riches
Plafond du Lift	5	Pour éviter des suggestions absurdes sur des mots très rares

Chapitre 3

Conception, implémentation et résultats

3.1 L'organisation globale de notre code

Pour que notre code reste lisible et facilement maintenable, nous avons fait le choix classique de séparer notre projet en plusieurs couches de responsabilités distinctes.

3.1.1 Les grandes pièces du puzzle

Notre programme Java s'articule autour de quatre grands dossiers :

1. **La gestion des données** (`DatasetService`) : c'est le lecteur de base. Il prend notre fichier texte (`dataset.txt`) et le transforme en données compréhensibles par le reste du code.
2. **Les objets manipulés** (le package `model`) : on y trouve nos classes de base, notamment `Transaction` (qui représente une simple phrase) et `RegleAssociation` (qui stocke une règle "Si A alors B").
3. **La logique métier** (`AprioriService` et `PredictionService`) : c'est le véritable moteur. C'est ici que les calculs de probabilités se font et que le système décide quels mots proposer.
4. **La communication avec l'extérieur** (`PredictionController`) : c'est l'interface qui permet à notre page web de venir discuter avec le moteur Java via une petite API REST.

3.1.2 Le parcours de la donnée

Pour faire simple, tout se passe en deux temps. D'abord, au moment où on lance le programme :

Texte brut $\xrightarrow{\text{Chargement}}$ Découpage en phrases $\xrightarrow{\text{Calculs Apriori}}$ Génération des règles $\xrightarrow{\text{Sauvegarde en mémoire}}$ Syst

Ensuite, à chaque fois qu'un utilisateur tape une lettre sur son clavier :

Texte tapé $\xrightarrow{\text{Nettoyage}}$ Recherche des derniers mots $\xrightarrow{\text{Comparaison aux règles}}$ Filtrage des scores $\xrightarrow{\text{Affichage}}$ Top-5

3.2 Plongée dans le code Java et Spring Boot

3.2.1 Comment représenter une règle ?

Comme expliqué plus tôt, nous avons créé une classe `RegleAssociation`. Elle permet de garder en mémoire l'antécédent (le bout de phrase de départ), la conséquence (le mot qui devrait suivre), et les fameuses trois notes (support, confiance, lift) qu'on a évoquées dans le chapitre précédent.

```

1 public class RegleAssociation {
2
3     private final List<String> antecedent; // Ex : ["je", "ne"]
4     private final String consequent; // Ex : "sais"
5     private final double support; // freq(A U B) / N
6     private final double confiance; // freq(A U B) / freq(A)
7     private final double lift; // confiance / P(B)
8
9     @Override
10    public String toString() {
11        return antecedent + " => " + consequent
12            + " [sup=" + String.format("%.4f", support)
13            + " conf=" + String.format("%.4f", confiance)
14            + " lift=" + String.format("%.2f", lift) + " ]";
15    }
16 }

```

Listing 3.1 – Modèle `RegleAssociation.java`

3.2.2 La lecture du fichier de base

Notre service `DatasetService` vient lire le fichier texte qui nous sert de base d'apprentissage. Fait important : on a décidé de ne pas supprimer les "petits mots" (les articles, pronoms, etc.) qu'on enlève souvent en Data Mining. Pourquoi ? Parce qu'en français, un "ne" ou un "pas" change tout le sens de la phrase et aide énormément à deviner la suite.

```

1 while ((ligne = reader.readLine()) != null) {
2     ligne = ligne.trim().toLowerCase();
3     String[] tokens = ligne.split("\\s+");
4     List<String> mots = new ArrayList<>();
5     for (String token : tokens) {
6         // Garder uniquement les mots d'au moins 2 caracteres
7         if (token.length() >= 2) {
8             mots.add(token);
9         }
10    }

```

```
10     }
11     // Une transaction doit avoir au moins 2 mots
12     if (mots.size() >= 2) {
13         transactions.add(new Transaction(mots));
14     }
15 }
```

Listing 3.2 – Extrait de DatasetService.java

3.2.3 Le moteur Apriori programmé maison

La plus grosse difficulté technique a été d'écrire nous-mêmes l'algorithme Apriori en Java, sans utiliser de bibliothèque toute faite. Voici comment on a procédé :

Première passe — Isoler les mots récurrents :

On commence par lister chaque mot. Si un mot apparaît moins de 3 fois dans tout le fichier, on l'ignore. C'est une façon d'évacuer les erreurs de frappe ou les mots beaucoup trop rares.

Deuxième passe — Construire les paires (Bigrammes) :

Ensuite, on assemble ces mots pour en faire des paires consécutives. Si les deux mots de la paire avaient survécu à la première étape, on compte la paire. Sinon, on passe notre chemin.

```
1  for (Transaction t : transactions) {
2      List<String> mots = t.getMots();
3      Set<String> pairesVues = new HashSet<>();
4      for (int i = 0; i < mots.size() - 1; i++) {
5          String w1 = mots.get(i);
6          String w2 = mots.get(i + 1);
7          // Pruning Apriori : les deux mots doivent être fréquents
8          if (!L1.contains(w1) || !L1.contains(w2)) continue;
9          String cle = w1 + "|||" + w2;
10         if (pairesVues.add(cle)) {
11             comptesBi.merge(cle, 1, Integer::sum);
12         }
13     }
14 }
15 // Filtrage par minSupport
16 Map<String, Integer> L2 = new HashMap<>();
17 for (Map.Entry<String, Integer> e : comptesBi.entrySet()) {
18     if (e.getValue() >= MIN_SUPPORT_COUNT) {
19         L2.put(e.getKey(), e.getValue());
20     }
21 }
```

Listing 3.3 – Génération des bigrammes — AprioriService.java

Troisième passe — Former des trios (Trigrammes) :

On répète le procédé pour faire des groupes de trois mots. On vérifie d’abord que les deux petits morceaux de la paire existent bien, pour gagner du temps de calcul.

```

1 // Pruning Apriori : les deux sous-bigrammes doivent être fréquents
2 if (!L2.containsKey(w1 + "||" + w2)) continue;
3 if (!L2.containsKey(w2 + "||" + w3)) continue;

```

Listing 3.4 – Pruning Apriori pour les trigrammes

Dernière étape — La création des règles :

Une fois qu’on a compté tout ça, on transforme nos groupes en véritables règles prédictives. On fait le tri, on élimine celles qui sont trop peu probables (en dessous de 10% de confiance) et on trie ce qui reste en multipliant la confiance par le lift.

$$\text{score final} = \text{confiance} \times \text{lift} \quad (\text{du meilleur au pire}) \quad (3.1)$$

3.2.4 Prendre la décision finale

Quand l’utilisateur écrit quelque chose, c’est le PredictionService qui entre en scène. Il va essayer d’être le plus intelligent possible :

1. Il nettoie ce qui a été tapé (enlève les majuscules, la ponctuation bizarre).
2. Il regarde en priorité les deux derniers mots écrits pour voir s’il connaît une règle de trigramme qui colle parfaitement.
3. S’il ne trouve rien, il se rabat sur le dernier mot écrit (règle de bigramme).
4. En dernier recours, il fouille dans toutes ses règles pour voir s’il y a un mot qui traîne.
5. Il retire bien sûr les mots que la personne vient juste de taper (pour ne pas radoter).
6. Enfin, il renvoie les 5 meilleures idées trouvées.

```

1 // Score = confiance x lift plafonne à 5
2 double score = regle.getConfiance() * Math.min(regle.getLift(),
3         5.0);
4 // Bonus x1.5 pour les regles trigramme (plus précises)
5 if (ant.size() == 2) score *= 1.5;
6
7 scores.merge(suggestion, score, Math::max);

```

Listing 3.5 – Scoring et sélection des suggestions — PredictionService.java

3.3 Mise en ligne et interface visuelle

3.3.1 Créer une API REST

Pour que notre moteur soit utilisable par n'importe quel site ou application mobile, nous l'avons encapsulé dans un contrôleur REST. Ça donne trois "portes d'entrée" principales :

TABLE 3.1 – Les points d'accès de notre API

Adresse	Méthode	Ce que ça fait
/api/predict?texte=...	GET	Nous renvoie nos fameux 5 mots pour compléter la phrase
/api/regles?limit=N	GET	Permet de fouiller dans les règles générées (très pratique pour nous lors des tests)
/api/stats	GET	Donne un petit résumé de la santé du modèle (combien de règles il a en tête, etc.)

L'annotation `@CrossOrigin("*")` est présente pour permettre à notre page web de venir interroger l'API sans se faire bloquer par le navigateur.

3.3.2 Le chargement en arrière-plan

L'un des trucs qu'on a mis en place, c'est que l'application calcule toutes les règles au moment même où on la démarre (grâce à un petit `@EventListener`). Comme ça, l'algorithme travaille une bonne fois pour toutes. Quand un utilisateur vient demander un mot, la réponse est déjà en mémoire, ce qui garantit une réponse quasi immédiate.

3.3.3 Le site web (Front-end)

Nous avons aussi codé une petite page web très basique en HTML, CSS et JavaScript. Il y a un grand champ de texte, et dès qu'on y tape quelque chose, un petit script vient interroger notre API en arrière-plan pour faire apparaître les suggestions sous forme de jolis boutons cliquables.

3.4 Bilan et analyse de nos résultats

3.4.1 Quelques chiffres

Après avoir laissé le programme lire notre fichier texte d'entraînement, on a jeté un œil à ce qu'il a pu construire :

TABLE 3.2 – Bilan chiffré du modèle

Info	Résultat
Règles créées en mémoire	Environ 1 500 règles prêtes à l'emploi
Seuil minimal pour exister	Vu au moins 3 fois dans le texte
Confiance plancher	10% minimum
Recette utilisée	Des paires et des trios en mode "séquentiel"

3.4.2 Tests en conditions réelles

Pour vérifier si tout cela tenait la route, nous nous sommes amusés à taper quelques débuts de phrases classiques. Le rendu est assez convaincant :

TABLE 3.3 – Test des prédictions

Si je tape...	Il me propose...
je ne	<i>sais, peux, veux, comprends, supporte</i>
il faut	<i>que, dire, bien, faire, savoir</i>
c'est très	<i>bien, important, difficile, intéressant, simple</i>
nous avons	<i>besoin, vu, fait, développé, utilisé</i>

3.4.3 Les points forts et les limites de notre travail

Ce qui fonctionne super bien

Le fait de croiser l'algorithme Apriori avec les N-grammes nous a pas mal avantageés par rapport à une méthode purement classique :

- **Un bon tri naturel** : grâce au filtre de "support", on ne propose pas de mots bizarres issus d'une faute de frappe dans notre texte de base.
- **Une notation plus intelligente** : calculer le "lift" en plus de la confiance évite de suggérer "le", "la" ou "de" à tout bout de champ.

- **Une vraie fluidité** : le système est ultra réactif parce qu'il a tout appris par cœur au démarrage.

Là où on bloque encore

Bien sûr, notre programme est loin d'être parfait. Pendant le projet, on a bien remarqué quelques défauts :

- **On a la mémoire courte** : le modèle ne regarde jamais plus loin que les deux mots précédents. Il est incapable de relier la fin d'une très longue phrase à son tout début.
- **C'est très dépendant du texte source** : si on entraîne notre modèle avec un livre de cuisine, il nous proposera des recettes de gâteaux même si on tape un mail professionnel.
- **Il est "bête"** : le programme fait juste des maths. Il n'a aucune idée de ce que les mots signifient réellement.
- **Le démarrage est un peu lourd** : si on lui donne un texte énorme à digérer, le lancement de l'application peut mettre un certain temps avant que tout soit mis en cache.

Ce qu'on aimerait faire pour la suite

Si on avait plus de temps à consacrer à ce projet, voilà ce qu'on essayerait de mettre en place :

- Utiliser une version plus rapide d'Apriori, comme l'algorithme **FP-Growth**, qui gère mieux les gros volumes de données.
- Rajouter une vraie compréhension des mots grâce au *Word Embedding* (comme Word2Vec).
- Pousser l'analyse à 4 ou 5 mots en arrière pour avoir des phrases plus cohérentes sur la longueur.
- Faire en sorte que le modèle apprenne spécifiquement du style d'écriture de la personne qui l'utilise en temps réel.

Conclusion et perspectives

En arrivant au bout de ce projet, notre objectif principal a été rempli : réussir à construire de nos propres mains un moteur capable de deviner le mot suivant. On a pris l'algorithme Apriori, on l'a un peu tordu pour qu'il comprenne l'ordre des mots, et on l'a couplé avec des N-grammes (paires et trios). Le tout fonctionne sur une application Java complète et peut être testé depuis un navigateur.

D'un point de vue personnel, ce travail a été une belle opportunité pour manipuler concrètement les concepts un peu abstraits vus en cours de Data Mining. Mettre les mains dans le cambouis pour coder soi-même le support, la confiance et le lift permet vraiment de démystifier la façon dont on extrait de l'information d'une masse de données. La plus grosse trouvaille reste l'adaptation de l'algorithme d'origine, qui ne se souciait pas de l'ordre des choses, pour en faire un vrai outil d'analyse séquentielle.

Dans les faits, le résultat est très satisfaisant. Le serveur démarre, fait ses calculs dans son coin, et une fois que l'utilisateur commence à taper, les réponses fusent sans aucun délai. Les règles qu'il a trouvées toutes seules, juste en lisant du texte, sont globalement pertinentes et ressemblent beaucoup à ce qu'un être humain pourrait proposer.

Nous restons persuadés que l'idée de lier le Data Mining classique au traitement de texte a encore de beaux jours devant elle. Surtout pour les petits projets ou les entreprises qui n'ont pas les moyens de faire tourner d'énormes intelligences artificielles. C'est une approche à la fois fiable, transparente et économe.

Évidemment, on sait que l'outil a ses limites. Mais les idées d'amélioration ne manquent pas. Entre l'optimisation des algorithmes de base avec du FP-Growth, l'ajout d'une pincée d'IA sémantique, ou encore l'analyse de contextes bien plus longs, le terrain de jeu reste immense pour aller encore plus loin dans l'autocomplétion.

Bibliographie

- AGGARWAL 2018 :** AGGARWAL Charu C., *Machine Learning for Text*, New York : Springer, 2018.
- AGRAWAL 1994 :** AGRAWAL Rakesh, SRIKANT Ramakrishnan, « Fast Algorithms for Mining Association Rules », in : *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB '94)*, San Francisco : Morgan Kaufmann, 1994, pp. 487–499.
- JURAFSKY 2024 :** JURAFSKY Dan, MARTIN James H., *Speech and Language Processing*, 3^e éd., Upper Saddle River : Pearson, 2024.
- MANNING 1999 :** MANNING Christopher D., SCHÜTZE Hinrich, *Foundations of Statistical Natural Language Processing*, Cambridge (MA) : MIT Press, 1999.
- MIKOLOV 2013 :** MIKOLOV Tomáš *et al.*, « Distributed Representations of Words and Phrases and Their Compositionality », in : *Advances in Neural Information Processing Systems (NIPS)*, vol. 26, 2013, pp. 3111–3119.
- PIATETSKY-SHAPIRO 1991 :** PIATETSKY-SHAPIRO Gregory, « Discovery, Analysis, and Presentation of Strong Rules », in : *Knowledge Discovery in Databases*, Cambridge (MA) : AAAI/MIT Press, 1991, pp. 229–238.
- TAN 2018 :** TAN Pang-Ning, STEINBACH Michael, KUMAR Vipin, *Introduction to Data Mining*, 2^e éd., Hoboken : Pearson, 2018.
- VASWANI 2017 :** VASWANI Ashish *et al.*, « Attention Is All You Need », in : *Advances in Neural Information Processing Systems (NIPS)*, vol. 30, 2017, pp. 5998–6008.
- ZAKI 2014 :** ZAKI Mohammed J., MEIRA Wagner Jr., *Data Mining and Analysis : Fundamental Concepts and Algorithms*, Cambridge : Cambridge University Press, 2014.

Annexes

Annexe A — Structure complète du projet

```
prediction-mots-ngram/
|-- src/
|   '-- main/
|       |-- java/org/prediction/
|           |-- PredictionApplication.java    (point d'entree Spring Boot)
|           |-- controller/
|               '-- PredictionController.java (API REST)
|           |-- model/
|               |-- RegleAssociation.java     (entite regle)
|               |-- Transaction.java          (entite phrase)
|               '-- service/
|                   |-- AprioriService.java   (algorithme Apriori)
|                   |-- DatasetService.java   (chargement corpus)
|                   '-- PredictionService.java (moteur de prediction)
|       '-- resources/
|           |-- dataset.txt                   (corpus d'entrainement)
|           '-- static/
|               '-- index.html                (interface web)
'-- pom.xml
```

Annexe B — Exemple de réponse de l'API /api/stats

```
{
  "statut"      : "actif",
  "minConfiance" : "10%",
  "nombreRegles" : 1500,
  "algorithme"  : "Apriori sequentiel (bigrammes + trigrammes)",
  "minSupport"  : 3
}
```

Annexe C — Exemple de réponse de l'API /api/predict

Requête : GET /api/predict?texte=je+ne

["sais", "peux", "veux", "comprends", "supporte"]

Annexe D — Commande de lancement

Prérequis : Java 17, Maven 3.8+

mvn spring-boot:run

Serveur disponible sur : http://localhost:8080

Interface web : http://localhost:8080/index.html

API predict : http://localhost:8080/api/predict?texte=bonjour

Table des matières

Remerciements	ii
Sommaire	iii
Introduction	1
1 Contexte, problématique et état de l’art	2
1.1 Mieux comprendre la prédiction de mots	2
1.1.1 Où retrouve-t-on cette technologie ?	2
1.1.2 Les vrais défis à relever	2
1.2 Ce qui se fait déjà dans le domaine	3
1.2.1 L’ancienne méthode : les règles strictes	3
1.2.2 L’arrivée des probabilités	3
1.2.3 L’ère des réseaux de neurones	3
1.3 Pourquoi lier le Data Mining et le texte ?	4
1.3.1 Des paniers d’achats aux phrases	4
1.3.2 La notion d’ordre change tout	4
2 Fondements théoriques des algorithmes utilisés	5
2.1 Un regard plus théorique sur les N-grammes	5
2.1.1 C’est quoi exactement un N-gramme ?	5
2.1.2 La logique mathématique derrière tout ça	5
2.1.3 Le mur de la dispersion des données	6
2.2 Le fameux algorithme Apriori	6
2.2.1 D’où vient-il ?	6
2.2.2 Comment savoir si une règle est pertinente ?	6
2.2.3 Le fonctionnement par étapes de l’Apriori	7

2.3	Faire marcher l'Apriori avec du texte	8
2.3.1	L'importance de l'ordre	8
2.3.2	Les règles qu'on obtient au final	8
2.3.3	Nos petits réglages maison	8
3	Conception, implémentation et résultats	9
3.1	L'organisation globale de notre code	9
3.1.1	Les grandes pièces du puzzle	9
3.1.2	Le parcours de la donnée	9
3.2	Plongée dans le code Java et Spring Boot	10
3.2.1	Comment représenter une règle ?	10
3.2.2	La lecture du fichier de base	10
3.2.3	Le moteur Apriori programmé maison	11
3.2.4	Prendre la décision finale	12
3.3	Mise en ligne et interface visuelle	13
3.3.1	Créer une API REST	13
3.3.2	Le chargement en arrière-plan	13
3.3.3	Le site web (Front-end)	13
3.4	Bilan et analyse de nos résultats	14
3.4.1	Quelques chiffres	14
3.4.2	Tests en conditions réelles	14
3.4.3	Les points forts et les limites de notre travail	14
	Conclusion et perspectives	16
	Bibliographie	17
	Annexes	18